

Sprawozdanie z Laboratorium nr 3

Przedmiot: Integracja Systemów Informatycznych **Temat:** Integracja z zewnętrznymi API i przetwarzanie danych **Student:** Anna Godlewska **Data:** 15.04.2026

1. Cel laboratorium

Celem laboratorium była rozbudowa aplikacji o mechanizmy integracji z zewnętrznymi źródłami danych. W ramach prac zintegrowano aplikację z dwoma zewnętrznymi API: Open-Meteo oraz JSONPlaceholder. Skupiono się na nauce nawiązywania połączeń HTTP, pobierania, filtrowania, agregacji i transformacji danych w formacie JSON. Dodatkowym celem była implementacja wizualizacji pobranych danych (wykresy) oraz utworzenie własnego endpointu API zwracającego przetworzone informacje.

2. Realizacja zadań

Zadanie 1: Praca na gałęziach i przygotowanie struktury Pracę rozpoczęto od utworzenia i zintegrowania nowej gałęzi w systemie Git za pomocą polecenia `git checkout -b feature/external-api-integration`. Następnie, wykorzystując środowisko frameworka Django, wygenerowano nową aplikację poleceniem `python manage.py startapp external_data`. Do pliku wymagań projektu (`requirements.txt`) dodano niezbędne do realizacji zadań biblioteki, w tym moduł `requests` do obsługi zapytań HTTP oraz `matplotlib` do generowania wykresów.

Zadanie 2: Integracja z Open-Meteo API i wizualizacja danych Zaimplementowano logikę pobierającą prognozę pogody dla wybranego miasta na podstawie przekazanych współrzędnych geograficznych. Pobrane z API dane poddano odpowiedniej obróbce – wyekstrahowano wartości temperatur (`temperature_2m`) oraz odpowiadające im ramy czasowe (`time`) dla najbliższych 24 godzin. Na podstawie tych danych wygenerowano wykres obrazujący zmiany temperatury, który został zapisany jako plik statyczny i osadzony w widoku podstrony. Podczas implementacji zadbano o odporność aplikacji, stosując bloki `try-except` do obsługi potencjalnych błędów połączenia oraz walidując kody statusu HTTP.

Zadanie 3: Integracja z JSONPlaceholder API Kolejnym etapem było nawiązanie połączenia z testowym API JSONPlaceholder, skąd pobrano listę postów i użytkowników. Zwrócony ciąg danych w formacie JSON przefiltrowano, a następnie poddano agregacji, aby wyliczyć szczegółowe statystyki – między innymi dokładną liczbę postów przypadającą na konkretnego użytkownika. Przetworzone informacje zaprezentowano w aplikacji webowej w formie czytelnej listy.

Zadanie 4: Własny endpoint API W ramach utworzonej aplikacji `external_data` przygotowano autorski endpoint pod adresem `/api/weather-summary/`. Jego zadaniem jest sieciowe udostępnianie przetworzonych wcześniej danych zewnętrznych (np. wyliczonej średniej temperatury na najbliższy dzień). Zwraca on informacje w ustandaryzowanym formacie JSON, co pozwala na ich łatwą konsumpcję przez inne usługi.

3. Lista kontrolna (Checklist)

- ☒ Czy stworzono i wykorzystano nową gałąź `feature/external-api-integration`?
- ☒ Czy zaimplementowano pobieranie danych z co najmniej dwóch wybranych zewnętrznych API (np. Open-Meteo, JSONPlaceholder lub inne z listy public-apis)?
- ☒ Czy dane z zewnętrznych API są poddawane **obróbce** (filtrowanie, wycinanie zakresów, agregacja)?
- ☒ Czy zaimplementowano przynajmniej jedną formę **wizualizacji danych** (np. wykres temperatury)?

- ☒ Czy stworzono własny endpoint API zwracający przetworzone dane zewnętrzne?
- ☒ Czy obsługowano błędy połączenia (try-except / try-catch) oraz sprawdzono kody statusu HTTP?
- ☒ Czy zainstalowane biblioteki (`requests`, `matplotlib` / `axios` itp.) zostały dodane do pliku zależności (`requirements.txt` / `package.json`)?
- ☒ Czy pobrane i przetworzone dane są wyświetlane w czytelny sposób (tabele, wykresy, listy)?
- ☒ Czy każdy commit ma jasny opis (np. "Add weather data processing and charts")?
- ☒ Czy sprawozdanie zawiera zrzuty ekranu działających integracji oraz wygenerowanych wykresów?
(tutaj wstaw odpowiednie zrzuty ekranu)

4. Wnioski

Laboratorium pozwoliło na praktyczne opanowanie technik integracji aplikacji webowych opartych na Django z zewnętrznymi usługami sieciowymi. Zdobyto cenne umiejętności w zakresie konsumpcji danych z publicznych API RESTowych, bezpiecznego parsowania oraz transformacji odpowiedzi typu JSON. Równolegle poszerzono kompetencje w obszarze graficznej prezentacji danych za pomocą biblioteki `matplotlib` oraz budowania własnych, dedykowanych punktów końcowych zwracających ustrukturyzowane informacje. Prawidłowa obsługa wyjątków znacząco poprawiła stabilność i niezawodność przygotowanego rozwiązania.